Minor Project

Secure Multi-Tier Web Application Network on AWS

Step-by-step work document: VPC, subnets, security groups, and firewalls

1. Project Objective

Design and implement a secure AWS network using a VPC with public and private subnets, security groups, route tables, an Internet Gateway, and Network ACLs. A web server sits in the public subnet and a database server sits in the private subnet, reachable only through the web tier.

Architecture

Internet → Internet Gateway → Public Route Table → Public Subnet (Nginx Web Server / Bastion) → Private Route Table → Private Subnet (Database Server).

AWS Services Used

VPC, Public Subnet, Private Subnet, Internet Gateway, NAT Gateway, Route Tables, Security Groups, Network ACLs, EC2.

Configuration Summary

Item	Value
VPC CIDR	10.0.0.0/16 (mytier-vpc)
Public subnet	10.0.1.0/24 (public-subnet)
Private subnet	10.0.2.0/24 (private-subnet)
Web server	Amazon Linux 2023, t2/t3.micro, Nginx, public subnet
Database server	Amazon Linux 2023, t2/t3.micro, MariaDB, private subnet
Web security group	web-sg: 22 (My IP), 80 (0.0.0.0/0), 443 (0.0.0.0/0)
DB security group	db-sg: 3306 and 22, both sourced from web-sg
Key pair	Same key pair used for both instances (needed for the bastion hop)

2. Step 1: Create the VPC

1. Go to **VPC → Your VPCs → Create VPC**.
2. Choose **VPC only**.
3. Name: `mytier-vpc`.
4. IPv4 CIDR: `10.0.0.0/16`.

5. Leave everything else at default and click **Create VPC**.

3. Step 2: Create the Subnets

Go to **VPC** → **Subnets** → **Create subnet**. Both subnets can be added in the same screen with **Add new subnet**.

Name	VPC	Availability Zone	IPv4 CIDR
public-subnet	mytier-vpc	Any AZ, e.g. us-east-1a	10.0.1.0/24
private-subnet	mytier-vpc	Same AZ as public	10.0.2.0/24

Enable auto-assign public IP on the public subnet

1. Select `public-subnet` → **Actions** → **Edit subnet settings**.
2. Tick **Enable auto-assign public IPv4 address**. Save.

> **Note:** Leave auto-assign public IP OFF for private-subnet. Instances there should never receive a public IP.

4. Step 3: Create and Attach the Internet Gateway

1. Go to **VPC** → **Internet Gateways** → **Create internet gateway**.
2. Name: `mytier-igw`. Click **Create**.
3. Select it → **Actions** → **Attach to VPC** → choose `mytier-vpc`.

5. Step 4: Configure Route Tables

Public route table

1. **VPC** → **Route Tables** → **Create route table**. Name: `public-rt`, VPC: `mytier-vpc`.
2. Open it → **Routes tab** → **Edit routes** → **Add route**: destination `0.0.0.0/0`, target = `mytier-igw`. Save.
3. **Subnet associations tab** → **Edit subnet associations** → tick `public-subnet`. Save.

Private route table

1. Create another route table: `private-rt`, same VPC.
2. Leave its routes as the default **local** route only — no route to the internet gateway. This is what keeps the subnet private.
3. **Subnet associations** → tick `private-subnet`. Save.

6. Step 5: Create Security Groups

Go to **EC2** → **Security Groups** → **Create security group**, in `mytier-vpc` for each.

web-sg (web server / bastion)

Type	Port	Source	Purpose
SSH	22	My IP	Remote login to the server
HTTP	80	0.0.0.0/0	Public access to the website
HTTPS	443	0.0.0.0/0	Reserved for SSL (not configured in this project)

db-sg (database server)

Type	Port	Source	Purpose
MYSQL/Aurora	3306	web-sg (security group, not an IP)	Database access from the web tier only
SSH	22	web-sg (security group, not an IP)	Bastion hop from the web server

Make sure both security groups keep the default **outbound rule (All traffic, 0.0.0.0/0)**. Removing it by mistake blocks all outbound traffic, including the bastion hop and any internet access.

7. Step 6: Launch the Web and Database Servers

web-server (public subnet)

1. **EC2 → Launch instance**. Name: `web-server`.
2. AMI: **Amazon Linux 2023**. Type: **t3.micro**.
3. Key pair: select one, i.e. `connect`. Used the same key in the database server.
4. **Network settings → Edit**: VPC `mytier-vpc`, Subnet `public-subnet`, Auto-assign public IP **Enable**, Firewall **Select existing security group** → `web-sg`.
5. Launch, then copy its **Public IPv4 address** once running.

db-server (private subnet)

1. **Launch instance** again. Name: `db-server`.
2. Same AMI and instance type.
3. **Key pair: the exact same key pair as web-server.** For the SSH bastion hop.
4. **Network settings → Edit**: VPC `mytier-vpc`, Subnet `private-subnet`, Auto-assign public IP **Disable**, Firewall **Select existing security group** → `db-sg`.
5. Launch, then copy its **Private IPv4 address** once running (it has no public IP by design).

8. Step 7: Connect via the Bastion Host

The database server has no public IP, so it can only be reached by first connecting to the web server and hopping from there over SSH.

Connect to the web server

1. Use **EC2 Instance Connect** (browser) or your own SSH client with the key pair file.
2. From your own computer:

```
ssh -A -i connect.pem ec2-user@<web-server-public-ip>
```

Windows (PowerShell) note: use the SSH agent

On Windows, -A alone does not load a key into a forwarding agent. The key must be loaded first:

```
# In an Administrator PowerShell window (one-time setup):
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent

# In a normal PowerShell window, from the folder with the key file:
ssh-add connect.pem
ssh -A ec2-user@<web-server-public-ip>
```

Hop to the database server

From the web server's own prompt, connect onward using the database server's private IP:

```
ssh ec2-user@<db-server-private-ip>
```

A successful hop lands on the database server's own prompt, for example `[ec2-user@ip-10-0-2-125 ~]$`. This proves the bastion setup and the db-sg SSH rule are both working.

9. Step 8: Add a NAT Gateway for Outbound Internet Access

The private subnet has no route to the internet, so package installs such as `dnf install` will time out on the database server until outbound-only internet access is added. A NAT Gateway provides this without exposing the subnet to inbound traffic.

1. **EC2 → Elastic IPs → Allocate Elastic IP address**. Leave defaults, click **Allocate**.
2. **VPC → NAT Gateways → Create NAT gateway**. Name: `mytier-nat`. Subnet: **public-subnet** (a NAT gateway always lives in a public subnet). Connectivity type: **Public**. Elastic IP: select the one just allocated. Create.
3. Wait until its state shows **Available** (1–2 minutes).
4. **VPC → Route Tables → private-rt → Routes tab → Edit routes → Add route**: destination `0.0.0.0/0`, target **NAT Gateway** → `mytier-nat`. Save.

NAT Gateway has a small hourly charge plus a per-GB data fee. So I will delete it during clean-up along with the other resources.

10. Step 9: Install Nginx on the Web Server

Run on `web-server`:

```
sudo dnf install nginx -y
sudo systemctl enable --now nginx
sudo systemctl status nginx     # should show: active (running)
curl localhost                  # prints the default Nginx HTML
```

Open **http://34.205.156.81** in a browser and confirm the page loads.

11. Step 10: Install MariaDB on the Database Server

Run on `db-server`, reached through the bastion hop:

```
sudo dnf install mariadb105-server -y
sudo systemctl enable --now mariadb
sudo systemctl status mariadb   # should show: active (running)
sudo mysql_secure_installation
```

Create a test database and a user restricted to the web tier's subnet:

```
sudo mysql -u root -p
CREATE DATABASE testdb;
CREATE USER 'webuser'@'10.0.1.%' IDENTIFIED BY 'YourStrongPassword!';
GRANT ALL PRIVILEGES ON testdb.* TO 'webuser'@'10.0.1.%';
FLUSH PRIVILEGES;
EXIT;
```

Allow remote connections, since MariaDB only listens on localhost by default:

```
sudo          sed          -i          's/^bind-address/#bind-address/'
/etc/my.cnf.d/mariadb-server.cnf
sudo systemctl restart mariadb
```

12. Step 11: Testing

This matches the project's testing checklist. Run each test and capture the result.

Test 1: Nginx is reachable from the internet

Open **http://<web-server-public-ip>** in a browser. The page should load.

Test 2: the database is not reachable from the internet

The database server has no public IP, so there is nothing to connect to from outside the VPC. This is enforced by design (no public IP, no inbound route from the internet), not by a firewall rule that could be misconfigured.

Test 3: bastion access works

From your computer, SSH into `web-server`, then from its prompt SSH into `db-server`'s private IP. A successful hop lands on the database server's own prompt.

Test 4: database connectivity from the web server

Install the MySQL client on `web-server` and connect to the database using its private IP:

```
sudo dnf install mariadb105 -y
mysql -h <db-server-private-ip> -u webuser -p testdb
```

A successful connection reaches a `MariaDB [testdb]>` prompt. Confirm it with a couple of quick queries:

```
SHOW TABLES;
SELECT 1+1;
SELECT USER(), @@hostname;
```

The last query is useful for the report: it shows the connecting account and the database server's hostname together in one result.

13. Network ACLs

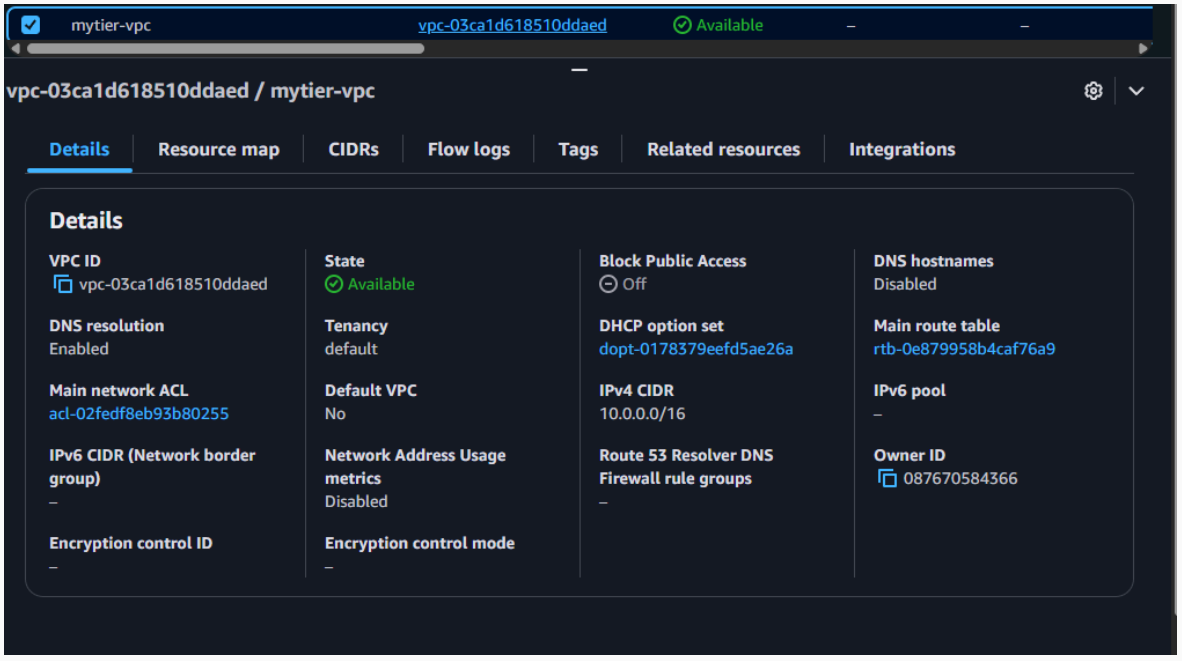
Network ACLs are stateless, so rules are needed for both directions. The default ACL (allow all) already satisfies the project's testing goals, since the security groups are what actually enforce the restrictions in this design. Adding explicit rules is a good addition to the report even if the default ACL is left in place for troubleshooting simplicity.

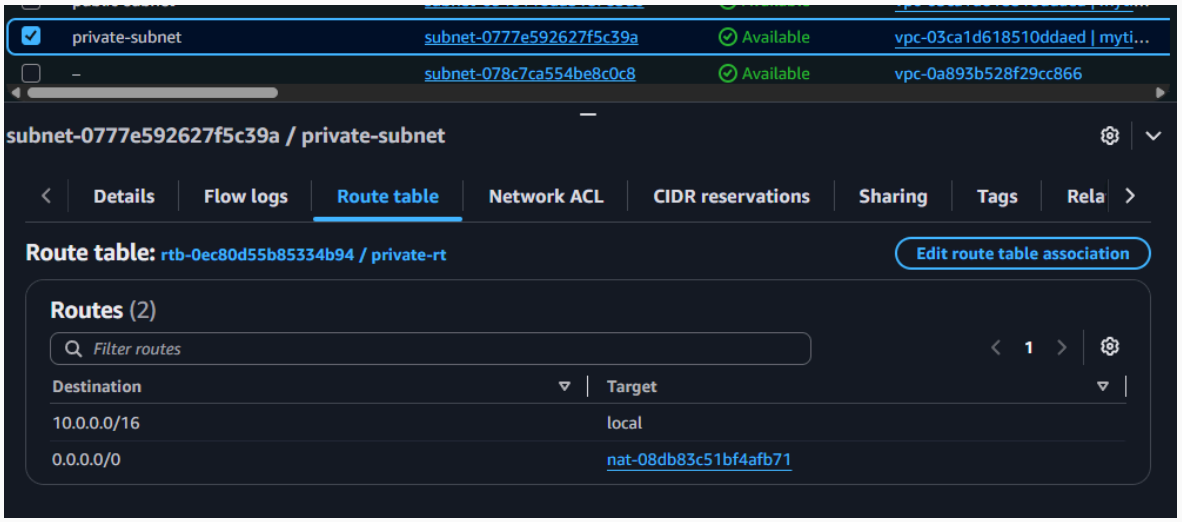
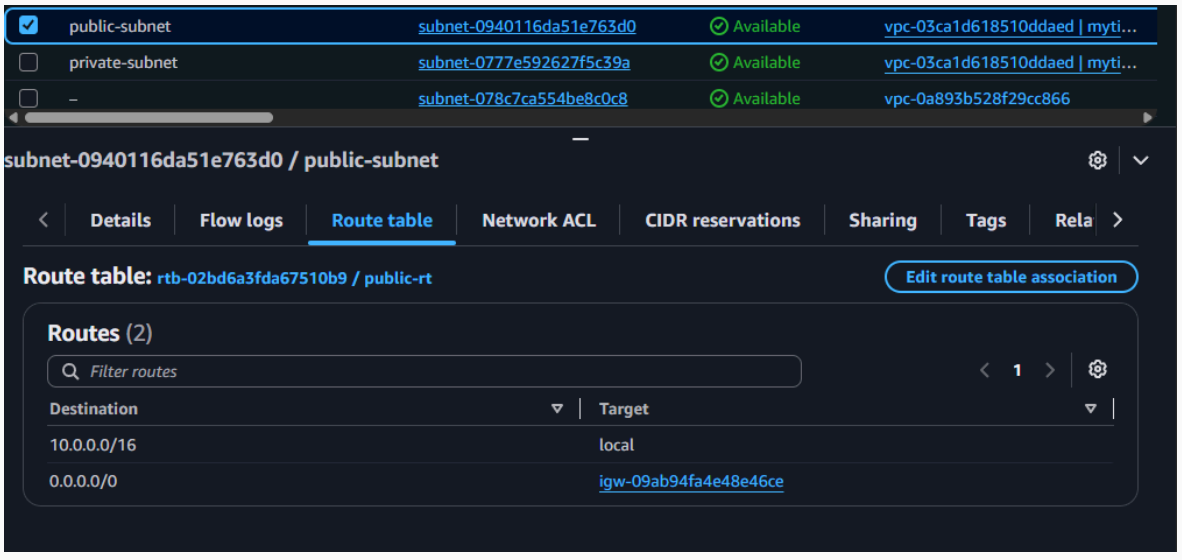
Subnet	Inbound (suggested)	Outbound (suggested)
Public subnet	22, 80, 443, and ephemeral ports (1024–65535) for return traffic	All, or mirror the same ports
Private subnet	22, 3306, and ephemeral ports, sourced from the VPC CIDR (10.0.0.0/16)	All, or mirror the same ports, plus 80/443 to the NAT Gateway route for updates

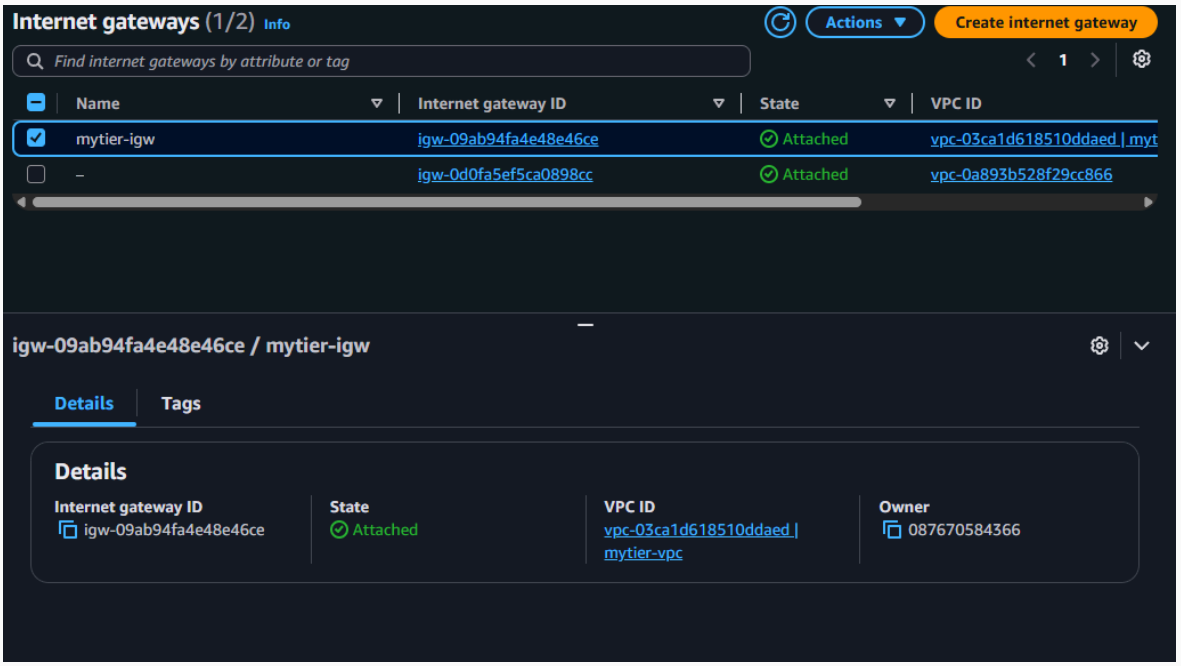
14. Deliverables: Screenshots

Capture the following for the report:

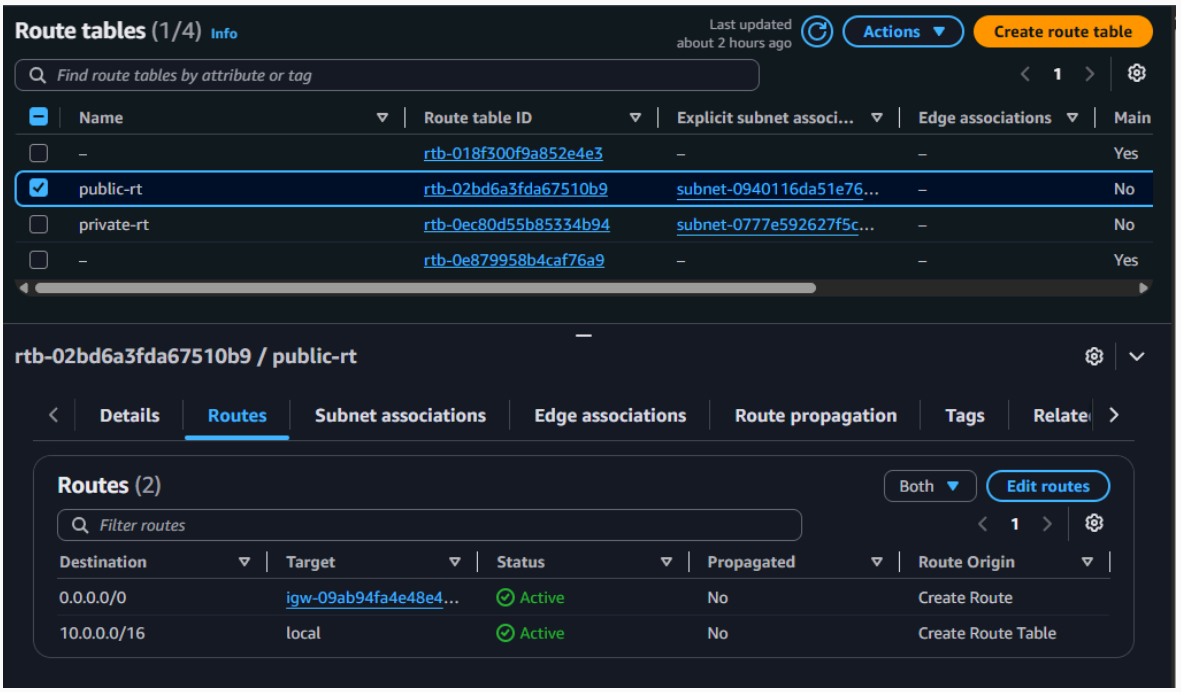
#	Screenshot	What to show
1	**VPC**	The VPC with its CIDR block (10.0.0.0/16)
2	**Subnets**	Both subnets with their route table associations visible

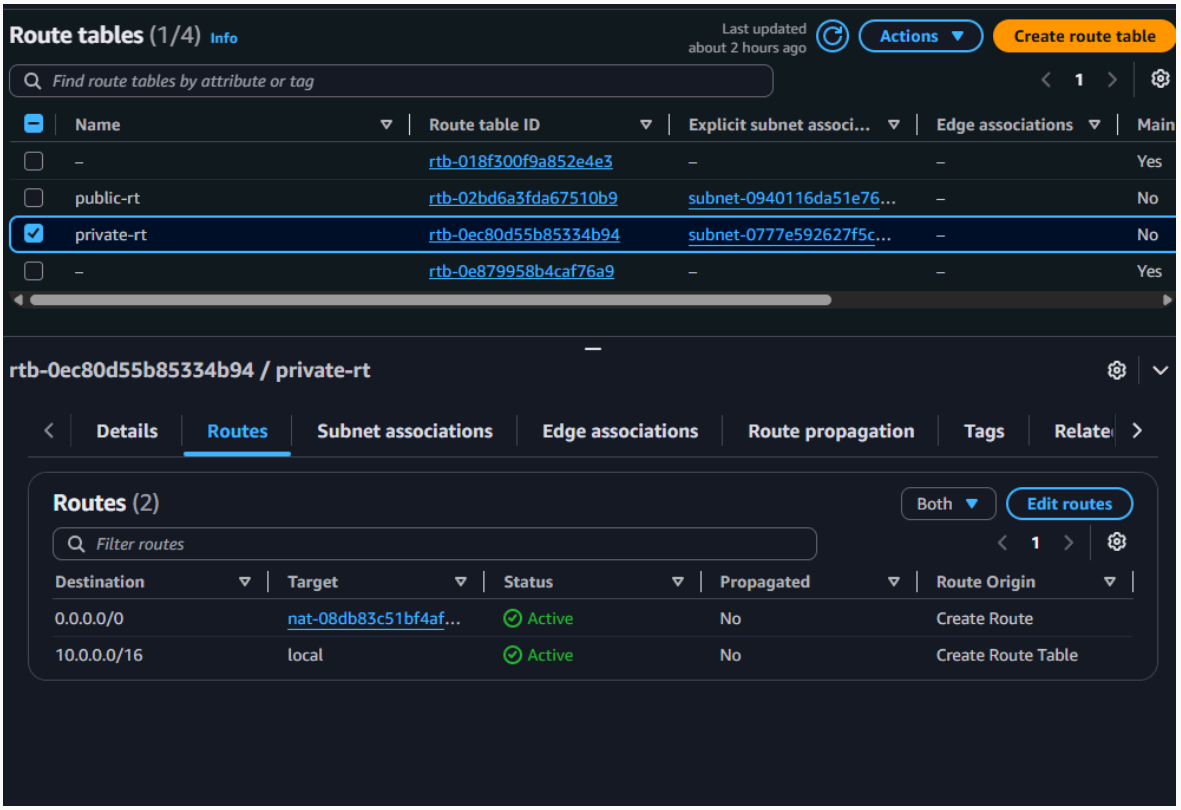


#	Screenshot	What to show
	 
3	**Internet Gateway**	Attached to the VPC

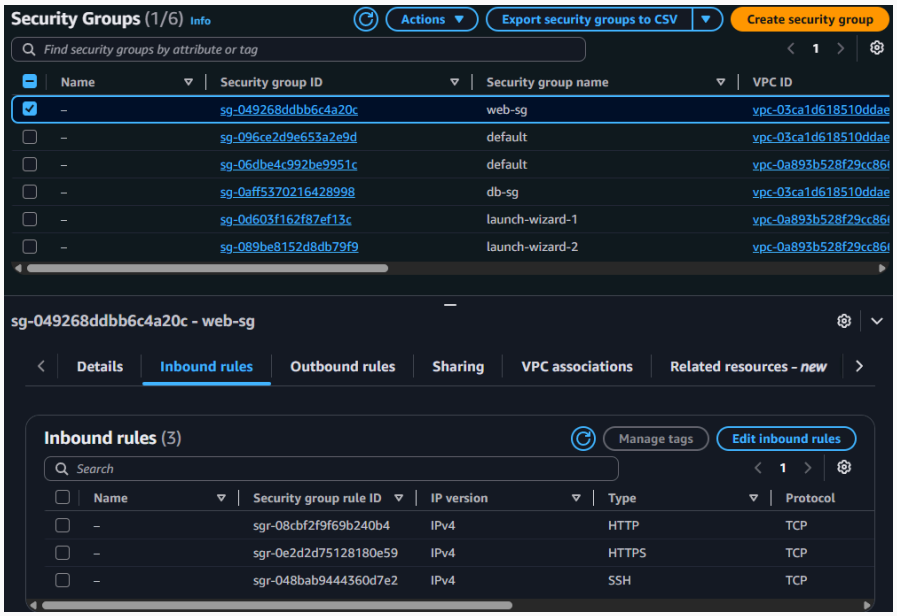
#	Screenshot	What to show
		
4	**Route tables**	Both public-rt and private-rt with their routes

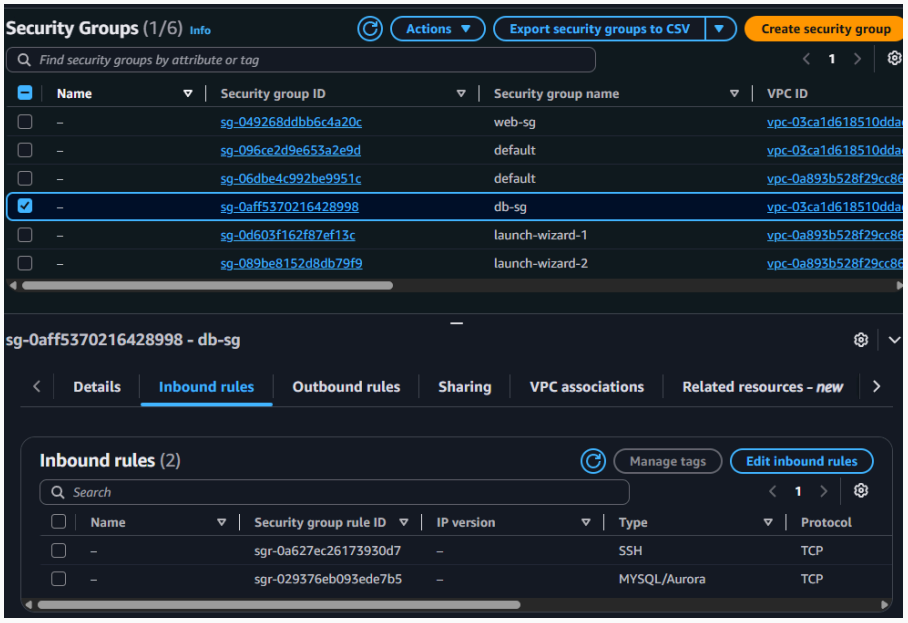
#	Screenshot	What to show



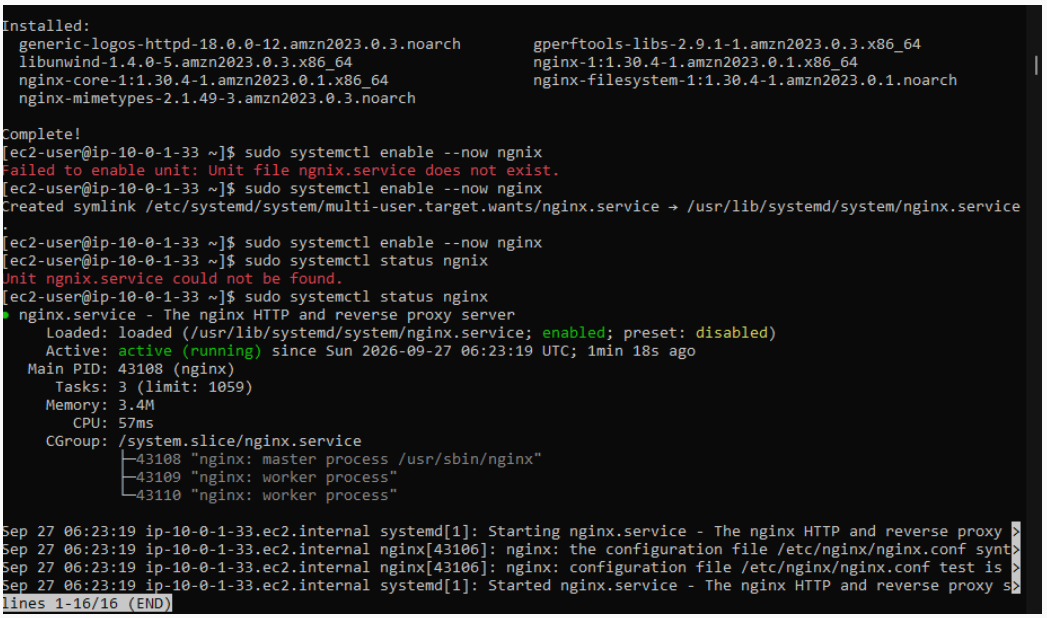
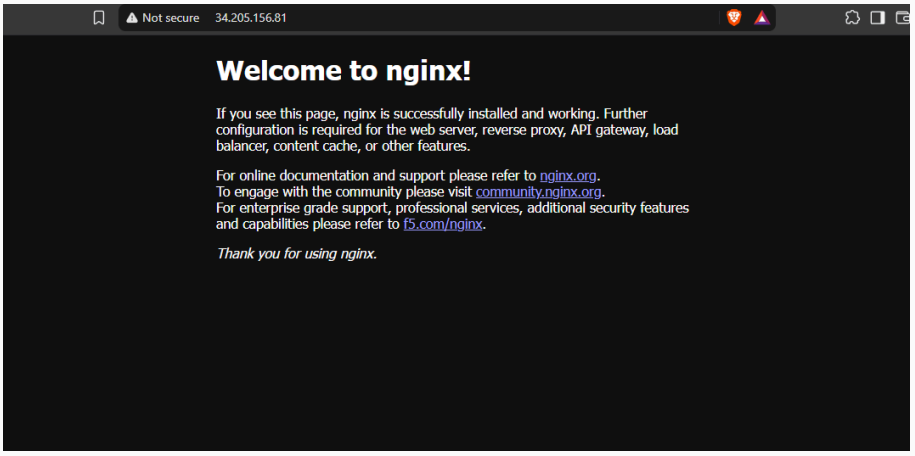


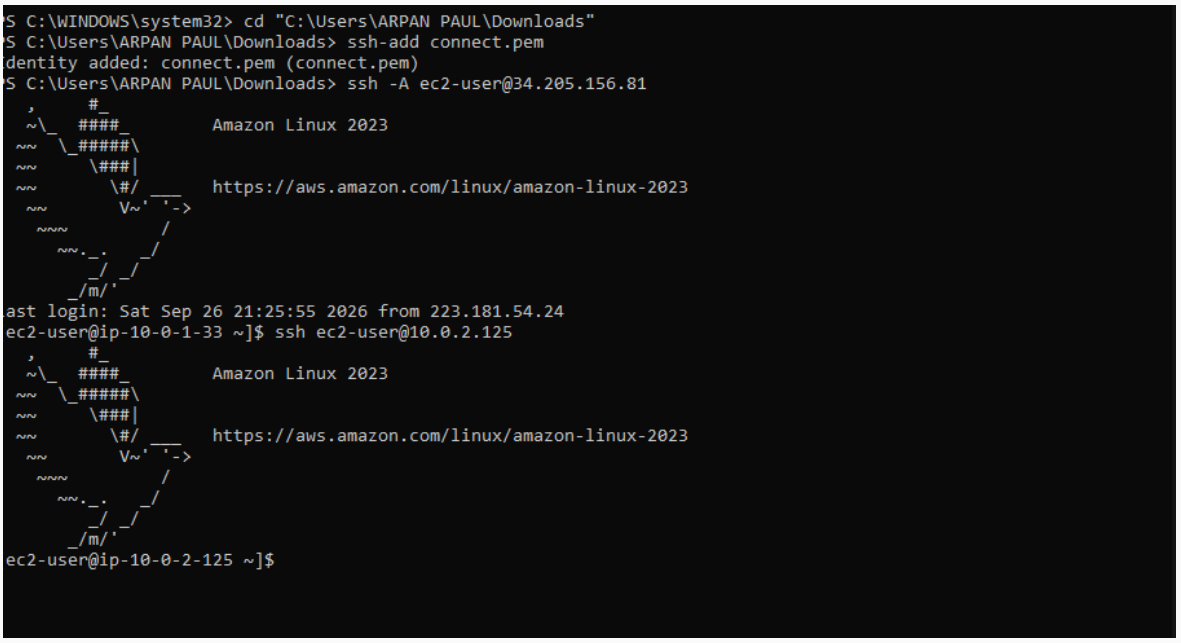
#	Screenshot	What to show
5	**Security groups**	Inbound rules for web-sg and db-sg



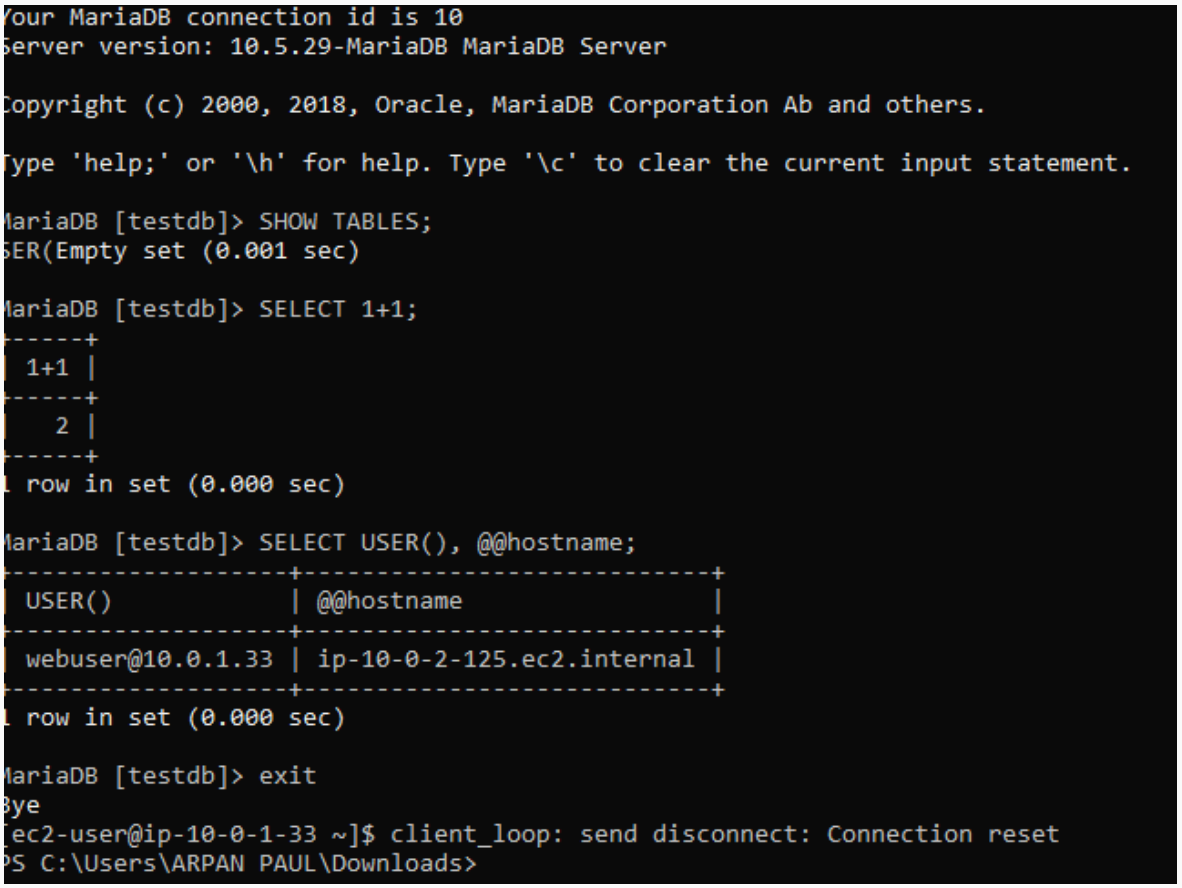


6	**Nginx website**	Browser showing the page at the web server's public IP

#	Screenshot	What to show
	 
7	**Bastion hop**	Terminal showing the SSH hop from web-server to db-server

#	Screenshot	What to show
		
8	**Database connection**	Terminal showing a successful mysql connection from web-server, including the USER()/@@hostname query

#	Screenshot	What to show



15. Troubleshooting

Problem	Fix
Security group rule accidentally added under Outbound instead of Inbound	Delete it from the wrong side and re-add it under Inbound rules. Then confirm Outbound still has the default All traffic / 0.0.0.0/0 rule — deleting the mistaken rule can leave outbound empty by accident, which blocks the bastion hop and all internet access from that instance.
"A security group description is required"	Click into the description field, select all and delete, then type a fresh description directly rather than pasting one in.
EC2 Instance Connect: "Failed to connect to your instance"	Retry after a minute, or connect with SSH from a local client using the key pair instead.
SSH hop times out (Connection timed out)	Check db-sg's inbound rule for port 22 is sourced from web-sg (by security group, not an IP). Also confirm web-server's own security group has its default outbound rule intact.

Problem	Fix
SSH hop gives "Permission denied (publickey...)"	Both instances must use the exact same key pair. On Windows, a key referenced with -i is not automatically forwarded; load it into the SSH agent first with ssh-add, then connect with ssh -A (no -i needed).
`dnf install` times out on the database server	The private subnet has no internet route by design. Add a NAT Gateway and route 0.0.0.0/0 from private-rt to it (Step 8).
Route table shows a `pl-...` destination instead of `0.0.0.0/0`	An autocomplete suggestion was selected while typing the destination. Edit the route, clear the field, type 0.0.0.0/0 fully, and click elsewhere without selecting a suggestion.
MariaDB: "Host ... is not allowed to connect"	The connecting host does not match any existing MariaDB user@host grant. Check `SELECT user, host FROM mysql.user;` and add a grant for the exact host or IP being used to connect.
MariaDB: "Access denied for user ... (using password: YES)"	The password typed does not match. Reset it with ALTER USER '...'@'...' IDENTIFIED BY '...'; and FLUSH PRIVILEGES;, then retry with the exact password.
`mysql: command not found` on the web server	The MySQL client was installed on the database server instead. Install it on the web server too: sudo dnf install mariadb105 -y.

16. Clean Up (to avoid charges)

1. **EC2 → Instances**: select both instances, then **Instance state → Terminate instance**.
2. **VPC → NAT Gateways**: select `mytier-nat`, **Actions → Delete NAT gateway**.
3. **EC2 → Elastic IPs**: select the address used by the NAT Gateway, **Actions → Release Elastic IP address**.
4. **VPC → Internet Gateways**: detach `mytier-igw` from the VPC, then delete it.
5. **VPC → Route Tables, Subnets, Security Groups**: delete the custom ones created for this project.
6. **VPC → Your VPCs**: delete `mytier-vpc` once everything inside it is removed.

17. Learning Outcomes

- Designing a VPC with public and private subnets
- Configuring route tables to control subnet-level internet access
- Using security groups (including group-to-group rules) to restrict access between tiers
- Setting up a bastion host pattern for reaching a private subnet
- Adding a NAT Gateway for outbound-only internet access from a private subnet
- Installing and testing a multi-tier application: a public web server and a private database server
- Understanding the role of Network ACLs alongside security groups